

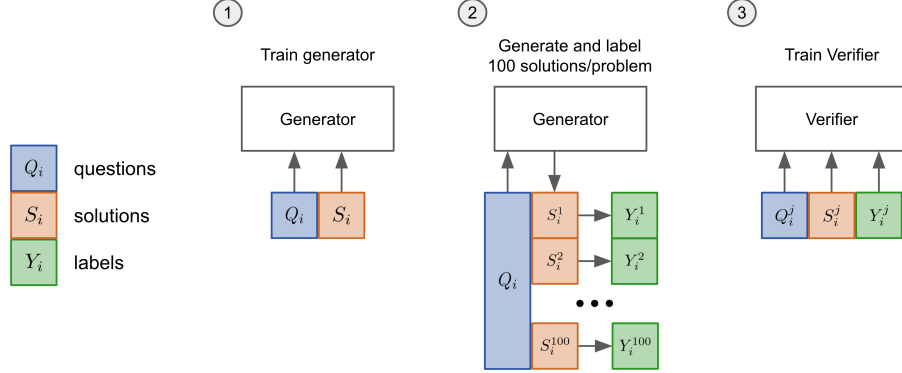


Figure 4: A diagram of the verification training pipeline.

training epochs. We use test@N to denote the percentage of problems solved correctly at least once when allowing the model to make N separate guesses for each problem. We use a low temperature ($T = 0$) to generate test@1 samples and we use a higher temperature ($T = 0.7$) to generate test@100 samples. Both temperature values were chosen empirically to produce the best results. Test@1 performance improves approximately monotonically, even though we quickly begin overfitting on test loss. Unfortunately, test@100 performance degrades much more sharply than test@1 as we increase the number of epochs. This is to be expected: as the model repeatedly encounters the same data, it becomes increasingly uncalibrated and overconfident in its predictions. At test time, this overconfidence leads to poor coverage of the solution space, an effect which only becomes noticeable when we are considering multiple samples at test time.

Choosing a model with good coverage is critical to successfully train verifiers. Empirically, we see that test@100 performance peaks within the first few epochs. For this reason, we use models trained for 2 epochs to generate samples for training verifiers. We provide several example solutions from 6B and 175B models in Appendix D. We also note that it is important to allow the model to generate the full natural language solution before outputting a final answer. If we instead finetune a 6B model to directly output the final answer without any intermediate steps, performance drops drastically from 20.6% to 5.2%.

4.2 Verification

To improve upon the finetuning baseline, we train verifiers to judge the correctness of model-generated solutions and search against these verifiers at test time. Conditioned on the problem and a candidate solution, the verifier outputs the probability that the solution is correct. Training solutions are labeled as correct or incorrect based solely on whether they reach the correct final answer. In practice, some solutions will reach the correct final answer using flawed reasoning, leading to false positives.

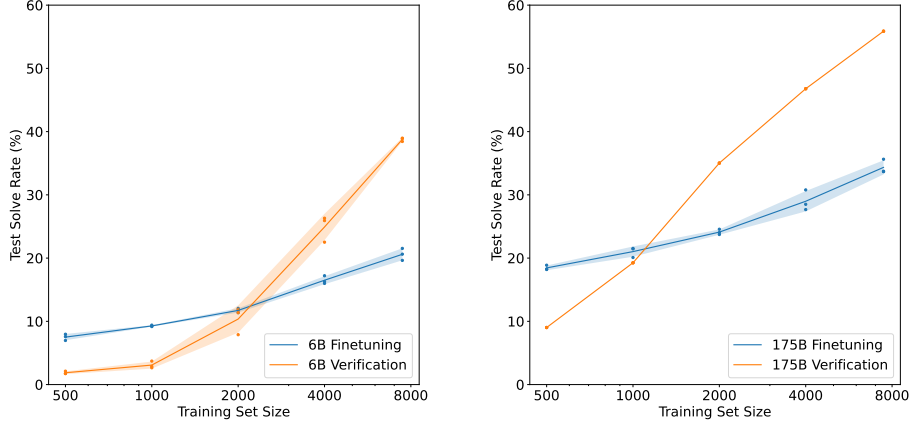


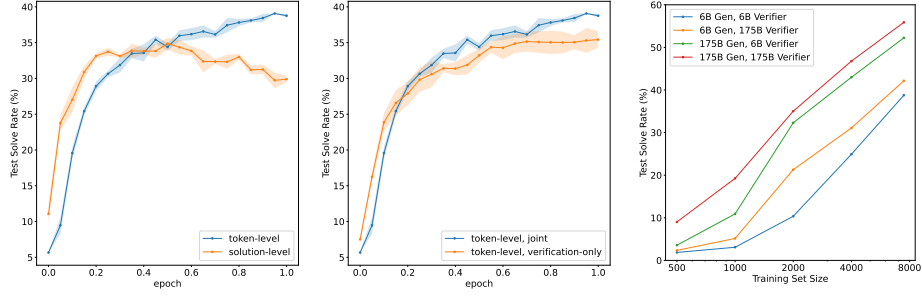
Figure 5: A comparison between finetuning and verification using 6B and 175B model sizes. Verification considers 100 solutions per problem. Mean and standard deviation is shown across 3 runs, except for 175B verification which shows only a single run.

As shown in Figure 4, we train the verifier as follows:

1. Finetune a model (the “generator”) for 2 epochs on the training set.
2. Sample 100 completions from the generator for each training problem and label each solution as correct or incorrect.
3. Train a verifier for a single epoch on this dataset.

Training for 2 epochs is enough for the generator to learn basic skills in this domain. We choose not to train for longer, since the diversity of generated solutions begins to collapse after this point, as shown in Figure 3. We train separate generator and verifier models to limit the generator’s training and prevent overfitting, but in principle, it should be possible to combine these models. Unless otherwise specified, we use the same model size for the generator and the verifier. In addition to predicting solution correctness, we also train the verifier with the same language modeling objective as the generator. This serves as a valuable auxiliary objective for the verifier. We discuss additional verifier training details in Appendix E.

At test time, we sample 100 completions to each test problem, rank them with the verifier, and then return the one with the highest verifier score. A comparison between verification and finetuning is shown in Figure 5 for both the 6B and 175B model sizes. We find that it is not beneficial to use verification at low dataset sizes. We believe this is due to the pressure to overfit to the correct answer: with small datasets, overfitting to the correct answer happens faster than learning more generalizable properties of correct reasoning. However, once we use a sufficiently large dataset, we see a strong boost from verifiers.



(a) Comparison between a verifier trained to predict correctness after every token (token-level) and one trained to predict correctness after only the final token (solution-level) (b) Comparison between a verifier trained jointly to predict correctness and perform language modeling (joint) and one trained only to predict correctness (verification-only) (c) Performance when varying the size of the generator and the verifier in isolation. Increasing the size of the generator has a larger impact than increasing the size of the verifier.

Figure 6: Verification ablations

It’s interesting to note that the 175B verifiers “take off” earlier than the 6B verifiers, requiring fewer training problems to surpass the finetuning baseline. See Appendix D for example solutions found by verifiers and Appendix F for a visualization of verifier confidence.

4.3 Verification Ablations

We can either train verifiers to make a single scalar prediction conditioned on the entire generated solution, or to make a scalar prediction after each token in the solution. By default, we choose the latter, training verifiers to make predictions after each token. This can be viewed as a token-level value function. We compare these two methods in Figure 6a, respectively labeled “solution-level” and “token-level”.

Predicting the value function at every token is a more challenging and noisier task than judging only the full completion. However, despite the initially slower training, the token-level verifier ultimately outperforms the solution-level verifier. Moreover, the token-level verifier is still improving late in training, whereas the solution-level verifier quickly shows signs of overfitting. We hypothesize that the full value function provides a useful auxiliary signal that encourages the model to judge the reasoning throughout solutions, rather than merely memorizing the correct final answer.

In Figure 6b, we ablate the objective used when training verifiers. As discussed in Section 4.2, we can optionally include a language modeling objective alongside the verification objective. We compare using both objectives to using only the verification objective. Although both are reasonable choices, including the language modeling objective is a strict improvement. This makes intuitive